Lani Voice Agent — Work Report

October 13 to 15, 2026

A. New Lani (V2) migration

1. find_resources lookup endpoint (the core of V2)

Issue: We're moving V2 Lani off the seven old question-by-question tools and onto a single lookup tool, find_resources, which did not exist yet. Without it, V2 can talk but cannot recommend real resources.

What I did: Built and deployed the endpoint, then validated it against live data with 9 test cards.

How I built it: api/find-resources.js on Vercel. It takes up to 3 typed needs (educator, class, tool, event, vendor, Utah REIA) and matches directly on GHL service keys, topics and levels, with no routing matrices. I reused the county/statewide/national geo scorer from vendors.js and the plain-words map from lani-catalog.js. It is committed and live at /api/find-resources.

How it works: Lani figures out the need with her own rules, calls this once, and gets back up to 2 matches per need with the exact name, a one-line description, and the cost. It enforces the rules we learned from past bugs: every vendor category matchable, specialty first, never fall through to unfiltered results, mark closest fit instead of silently swapping strategy, and score educators across all 44. It returns needs_location when a local service has no city or county, and flags out-of-state coverage. It gives an honest no_match instead of padding.

2. Clone the web call page for the new agent

Issue: We needed a separate web page pointed at the new V2 agent, with the live page left untouched.

What I did: Cloned public/index.html into public/index-new.html and repointed it to the V2 agent.

How I built it: Changed one constant, the assistant ID. Public key and GHL webhook stayed the same.

How it works: The new page drives the V2 agent, and the live page stays on the original agent.

3. Wire the V2 agent (tools and server URL)

Issue: The new agent had no tools attached, so it could talk but not route.

What I did: Copied the live tools onto V2 and confirmed its prompt stayed intact.

How I built it: A one-off script that patches the V2 model's tool list through the Vapi API without touching the system prompt. We later decided V2 runs on only find_resources, so this was superseded, but it confirmed the mechanics.

How it works: The server URL lives at the Vapi account level and every agent inherits it, so V2 already matched live with no per-agent change.

4. Two-chat sync rules and tracker infrastructure

Issue: We were running two chats on V2 at once and they edited it without seeing each other's plans, which caused a collision.

What I did: Adopted the sync rules we agreed on and built the tracker support they need.

How I built it: Added a planned event type and a lani_scope column (V1/V2/both) to the tracker through migrations.

How it works: Before any V2 change I log a planned row and tag which Lani it is for, and each session reads the other chat's last 48 hours first.

B. Call pipeline reliability and cost

5. End-of-call save recorded no calls

Issue: No voice call had saved to GHL or Supabase since August 13.

What I did: Found the root cause and hardened the save.

How I built it: Traced it to the save endpoint getting zero calls. V2 is tested in the Vapi dashboard with no server URL, so it never calls the endpoint, and the save code itself is fine. I added a longer timeout so the save cannot be cut off mid-write.

How it works: The save is ready and fills in as soon as we reconnect the Vapi server URL, which is on hold.

6. Call-completeness filter

Issue: Abrupt or incomplete calls, like voicemail, no-answer, and dropped calls, were firing the paid workflow and SMS.

What I did: Gated the workflow so only real, completed calls run through it.

How I built it: Two layers. A code gate in ghl-sync.js that fires only when the call is at least 20 seconds and the end reason is clean, plus a visible Completed Real Call Gate at the top of the Voice Agent Lead workflow.

How it works: Supabase still records every call; only the billed workflow and the SMS are gated. The payload now also carries the end reason and duration.

7. GHL to Supabase webhook sync fix

Issue: The lean event-registration webhooks were overwriting investor profiles with blanks on every event sign-up.

What I did: Fixed the merge and standardized all the branch bodies.

How I built it: Deployed the edge function with merge semantics so each branch only writes the fields it actually carries, fixed the contractor-speciality tag, and restored the auth check.

How it works: An event registration now updates only its own fields and leaves the rest of the profile intact.

C. CRM and workflow hardening

8. Vendor workflow hardening

Issue: A returning vendor who called back as an investor kept a stale Vendor Service Type value and got misrouted as a vendor, because the sync clears that field after the workflow has already fired.

What I did: Fixed the misroute and hardened both vendor workflows.

How I built it: Switched the branch to read the fresh inbound webhook value instead of the stored contact field, enabled Stop-on-Response, and added a do-not-disturb check before every SMS on both workflows.

How it works: Routing now uses a per-call signal, so returning callers are not misrouted, and no SMS goes to a do-not-disturb contact. Live verification waits on the server URL.

D. Tools delivered

9. Metrics dashboard

Issue: We had no live view of agent activity.

What I did: Built and deployed a live dashboard.

How I built it: A page at /dashboard.html reads a thin server-side proxy over a new read-only database function, so the service key stays on the server.

How it works: It shows total routings, 7-day and 21-day volume, educator match rate, routing by stage and action, top educators and tracks, and a voice-calls panel. The routing data is rich; the voice-call panel stays sparse until call-saving reconnects, and the page says so.

10. 72-hour follow-up SMS

Issue: There was no follow-up after a vendor referral.

What I did: Added a vendor-only follow-up branch.

How I built it: In the Voice Agent Lead workflow, when a vendor match exists: wait 72 hours, check do-not-disturb, then send an SMS asking if they connected.

How it works: It fires once vendor-match call data flows again, which is gated on the server URL.

E. Routing audits and design

11. Education routing matrix audit

Issue: We suspected systematic gaps in education coverage.

What I did: Audited it, then corrected my own first pass.

How I built it: Checked the matrix, then the code. 12 strategies have full coverage, 9 commercial sub-assets correctly collapse to commercial, and the 5 gaps I first flagged are already covered by fallbacks in the code.

How it works: Every strategy routes to a track, directly or through a fallback, so no fill is needed. Dedicated tracks are an optional future enhancement.

12. Intake and routing coupling gaps

Issue: Some signals we collect (goal, credit, capital) were not steering which rule gets selected, and a vendor primary need question never fired.

What I did: Verified one drift was already fixed and closed the rest against the V2 plan.

How it works: The high-value half, where signals steer which resources surface, already shipped. The rule-scoring half is moot under V2 because the old intake rules are being retired, so it is closing as sunset rather than waiting to be built.

13. Scored confidence matching (design only)

Issue: Lani decides close enough to recommend by judgment, which is inconsistent.

What I did: Drafted the design.

How it works (planned): Each candidate gets a 0 to 100 score from weighted overlap, with a configurable threshold and confidence bands. It becomes the ranking inside find_resources.

F. Supporting pieces

14. Vendor referral billing table

Detail: An append-only log, partners only, one row per delivered referral, non-billable when the same partner reached the same caller within 30 days. It feeds both billing and the least-recently-referred rotation in find_resources.

15. Vendor location matching

Detail: Synced the GHL Area Serviced field into Supabase as counties, statewide, and national flags, and added a county scorer that find_resources reuses.

16. Analysis reports

Detail: Three write-ups: a webhook cost audit across 131 workflows that found and disabled one runaway test workflow, a sync cost breakdown showing the webhook is the aggregate cost while the structured outputs are a per-call slice we are already cutting to Haiku 4.5, and a research doc on making Lani sound more natural.

Bottom line: V2's find_resources engine is live and tested. What is left before V2 reaches callers is the prompt rules, the go-ahead to attach the tool to a test agent, and the content prerequisites, which are enrolling the real partners and adding the educators. All of it is tracked.